PENTECOST UNIVERSITY

FESAC

Secure Login Authentication System with Two-Factor Authentication and Role-Based Access

Assignment Report

Name: Addo Eric

Student ID: PUIT24210065

Programme: Information Technology

Date: October 2026

Table of Contents

1. Introduction and objectives

2. Technologies used

3. System architecture

4. Database design

5. User identification and registration

6. Two-factor authentication

7. Sessions

8. Role-based authorization

9. Salt and pepper password protection

10. Other protections

11. Deleted accounts, backup and restore points

12. Reports and the password demonstration

13. Implementation details

14. Testing

15. Limitations and future improvements

16. Conclusion

17. Instructions for running the application

1. Introduction and objectives

Almost every application that stores personal information needs to answer two questions: *who is this user?* and *what are they allowed to do?* This project builds a small but realistic authentication system that answers both questions and protects the passwords it stores.

The objectives, taken from the assignment brief, were to implement:

1. **User identification and registration**: users create an account and later prove who they are.
2. **Two-factor authentication (2FA)**: a password alone is not enough; a one-time code is also required.
3. **Role-based authorization**: what a user may do depends on a role (`admin` or `student`) that the server enforces.
4. **Salt and pepper**: passwords are stored using a salted hash combined with a secret server-side pepper.

The system has a mobile and web interface built with Expo, and a PHP and MySQL backend that runs locally on XAMPP or Laragon. The same code is also deployed online at **https://puit24210065.vercel.app**, so that it can be tried from a public link.

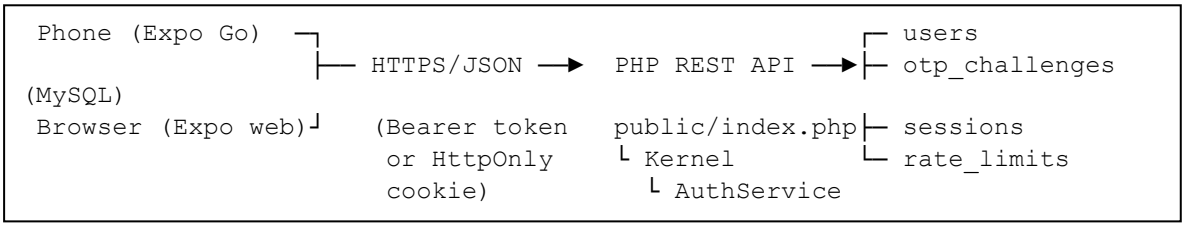
2. Technologies used

Layer	Technology	Why it was chosen
Interface	Expo, React Native, TypeScript	One code base for Android, iOS and the browser, as required by the brief.
Navigation	Expo Router	File-based routing; the same routes work on phone and web.
Token storage (phone)	Expo SecureStore	Uses the iOS Keychain and Android Keystore.
API	PHP 8.1+ (no framework, no Composer)	Runs on any XAMPP or Laragon installation without extra downloads.
Database	MySQL / MariaDB with PDO	Prepared statements, foreign keys, unique indexes.
Password hashing	`password_hash()` / `password_verify()`, Argon2id (bcrypt fallback)	Built into PHP, adaptive and salted.
Email	A small SMTP client written for the project	Sends the one-time code; no external library needed.
Online hosting	Vercel (static site + `vercel-php` functions)	Serves the Expo web build and runs the same PHP API under `/api`, over HTTPS.

Layer	Technology	Why it was chosen
Online database	Aiven MySQL 8.4	Managed MySQL reachable from Vercel, with an encrypted (TLS) connection.
Online email	Gmail SMTP with an App Password	Delivers the one-time codes on the public site.

Development tools: Visual Studio Code, Git and GitHub, and the AI coding assistant Claude Opus 5.5 (Anthropic), which was used to help write the code, tests and documentation.

3. System architecture



- **Client.** The Expo app has three signed-out screens (sign in, register, verify code) and three signed-in screens (dashboard, profile, administration). A route guard sends signed-out users to the sign-in screen, but this is only for convenience; the real checks are made by the server.
- **API.** `public/index.php` is the only entry point. The `Kernel` class handles routing, CORS, CSRF checks, session lookup and role checks, and returns response objects, which makes the whole API testable without a web server. `AuthService` contains the business logic (registration, password check, one-time code, sessions), `AdminService` the administrator features (deleting accounts, backup, restore points, reports, password demo) and `AuditLog` the event log. Other small classes handle passwords, tokens, rate limiting, validation and email.
- **Database.** All access uses PDO prepared statements. Timestamps are generated by the application in UTC so behaviour does not depend on the database server's time zone.
- **Configuration.** Secrets (pepper, database password, SMTP password) live only in `backend/.env` on the server. The server refuses to start if the configuration is unsafe, for example if the pepper is missing, CORS is a wildcard, or the development one-time-code mode is enabled in production.
- **Online version.** On Vercel, `api/index.php` simply includes `backend/public/index.php`, so the public site runs the same PHP code. The website and the API share one domain, so the session cookie stays first-party. Settings come from Vercel's encrypted environment variables instead of `.env`, the database is Aiven

MySQL (TLS verified against Aiven's CA certificate), and codes are emailed through Gmail.

4. Database design

Table	Purpose	Important columns and constraints
`users`	One row per account	`email` (unique), `password_hash`, `role` (`ENUM('student','admin')`, default `student`), `created_at`, `last_login_at`, `deleted_at` (set when an admin deletes the account)
`otp_challenges`	A pending or used second-factor challenge	`challenge_hash` (unique), `otp_hash`, `attempts`, `expires_at`, `used_at`; foreign key to `users` with `ON DELETE CASCADE`
`sessions`	Login sessions	`token_hash` (unique), `expires_at`, `revoked_at`; foreign key to `users`
`rate_limits`	Counters for abuse prevention	`bucket` (primary key, a hash), `hits`, `window_start`
`auth_events`	Log of security events for the reports	`event`, `user_id` (foreign key, `ON DELETE SET NULL`), `ip`, `detail`, `created_at`
`restore_points`	Saved configurations	`label`, `created_by`, `created_at`, `user_count`, `config` (each account's role and deleted status, as JSON)

Design decisions:

- The `email` column is unique, so the database itself rejects duplicates even if two requests race.
- Neither the one-time code, the challenge identifier, nor the session token is stored in readable form. Only hashes are stored (see sections 6 and 7).
- Indexes exist on foreign keys and on expiry columns, which are used when looking up sessions and cleaning old rows.
- Deleted accounts are not erased: `deleted_at` marks them, so the data stays in the database and can be restored.
- The complete script is `backend/database/schema.sql`. An existing database from the first version is upgraded with `backend/database/migrations/002_student_role_backup_reports.sql` (renames the role `user` to `student`, adds `deleted_at` and the two new tables).

5. User identification and registration

Registration (`POST /api/register`). The client validates the form first (name length, email format, password rules, matching confirmation) to give instant feedback. The server then validates the same rules again, because client checks can be bypassed. The password

must be 12 to 128 characters long, contain at least one letter and one digit, must not be a very common password, and must not equal the email address. The email is converted to lower case; if an account with that email already exists the server answers `409`. The role of a new account is always `student`; the request body cannot choose a role.

Sign-in, step 1 (`POST /api/login`). The server looks up the account by email and verifies the password. For an unknown email it still performs a hashing operation, so that response time does not reveal whether an account exists. A wrong password and an unknown email produce exactly the same response (`401`, "Invalid email or password."). At this point the user is **not** signed in: no session exists until the second factor is verified.

6. Two-factor authentication

After a correct password the server:

1. generates a six-digit code with `random_int()`, PHP's cryptographically secure generator;
2. creates a random 256-bit *challenge identifier* and returns it to the app;
3. stores only a keyed hash of the code (HMAC-SHA256, keyed with a secret derived from the pepper) and a hash of the challenge identifier;
4. delivers the code by email through the SMTP server configured in `.env`.

Sign-in, step 2 (`POST /api/verify-otp`). The app sends the challenge identifier and the code. The server enforces:

- **Expiry**: a code is valid for five minutes.
- **Attempt limit**: five wrong guesses lock the code. The attempt counter is increased with a single atomic SQL statement *before* the comparison, so parallel requests cannot gain extra guesses.
- **Single use**: the code is marked as used with `UPDATE ... WHERE used_at IS NULL`; only the request that changes the row succeeds, so a code cannot be replayed.
- **Replacement**: asking for a new code, or signing in again, invalidates the previous code.
- **Account-level limit**: wrong guesses are also counted per account across challenges, so repeatedly starting a new sign-in does not reset the guess budget.

Development mode. To demonstrate the system without an email server, `OTP_DELIVERY=dev` makes the API return the code in its response, and the app shows it in a clearly labelled "Development mode" box. The server refuses to start with this setting when `APP_ENV=production`, so the code can never be exposed by a production deployment. The setting is off unless it is written into `.env`.

7. Sessions

When the code is correct the server creates a random 256-bit session token and stores its **SHA-256 hash**, together with an expiry time (8 hours by default). If the database were leaked, the stored values could not be used as sessions.

- **Native app:** the token is returned once and kept in **Expo SecureStore** (Keychain / Keystore). It is sent as a `Bearer` header.
- **Web:** the token is **never given to JavaScript**. It is sent as an `HttpOnly` cookie (`Secure` when configured, with `SameSite`). `localStorage` is deliberately not used, because any script on the page could read it. Because browsers attach cookies automatically, the server also protects cookie-based state-changing requests against cross-site request forgery: logout requires an allowed `Origin` header and a custom `X-Client-Type` header.
- **Logout** sets `revoked_at` in the database, so the token stops working immediately, even if someone copied it.

8. Role-based authorization

There are two roles, `student` and `admin`. For every protected request the server finds the session, loads the user **from the database** and checks the role. The interface hides the administrator menu (Users, Backup, Reports, Password demo) from students, but that is only for usability: every admin endpoint (`/api/admin/...`) returns `403 Forbidden` to a student and `401` to a visitor without a session, whatever the client code or request contains. Changing a role in the database takes effect on the next request.

Public registration cannot create administrators. The first administrator is created on the server by registering normally and then running `php bin/set-role.php you@example.com admin`; only someone with access to the server can do this. Alternatively, the server setting `ADMIN_EMAILS` lists administrator email addresses. A listed account is promoted only after a successful login with both the password and the emailed code, which proves the person controls that inbox. The name a user types and the act of registering never grant the admin role. The online version uses this setting.

9. Salt and pepper password protection

- **Salt.** `password_hash()` generates a unique random salt for every password and stores it inside the hash string. Two users with the same password therefore get different hashes, and precomputed (rainbow) tables are useless.
- **Pepper.** A secret value, at least 32 characters, is kept in the server configuration, never in the database or the app. Before hashing, the password is passed through `HMAC-SHA256` keyed with the pepper, and the result is given to `password_hash()`. An

attacker who steals only the database cannot even start guessing passwords offline without the pepper. This pre-hashing step also avoids bcrypt's 72-byte limit.
- **Algorithm.** Argon2id is used when PHP supports it, otherwise bcrypt with cost 12. `password_needs_rehash()` upgrades old hashes at the next successful login.
- **Plaintext** passwords are never stored, logged or returned by the API.

The cost of the pepper is that it cannot be rotated without forcing password resets, because it is part of every hash. This is listed under limitations.

10. Other protections

- **Rate limiting** (stored in MySQL): 8 failed sign-ins per email and 30 requests per IP per 15 minutes; 30 code attempts per IP; at most 10 registrations per hour per IP; a 30-second cooldown between code requests. Responses are `429` with a `Retry-After` header.
- **Prepared statements** everywhere, so input cannot change the meaning of a query.
- **CORS** allows only the exact origins listed in `.env`, never `*`, because cookies are involved.
- **Safe errors.** Authentication failures are generic. Unexpected errors return a generic message while details go to the server log. Misconfiguration returns `500`/`503` without naming settings or values.
- **Response headers**: `Cache-Control: no-store`, `X-Content-Type-Options`, a restrictive `Content-Security-Policy`, and `Strict-Transport-Security` in production.

11. Deleted accounts, backup and restore points

Deleting an account. On the Users page an administrator can delete an account. The row is not removed: the server sets `deleted_at`, and the account is then shown as *deleted* (struck through) in the list and counted separately. A deleted account cannot log in (it gets the same "Invalid email or password." message as anyone else), its open sessions are ended at once and any pending code is cancelled. As a second line of defence, the session check itself also refuses any account marked as deleted. An administrator cannot delete their own account. A deleted account can be restored with the same data and password.

Full database backup. On the Backup page an administrator downloads the whole database as one SQL file (`GET /api/admin/backup`): for each of the six tables it contains `DROP TABLE`, `CREATE TABLE` and `INSERT` statements with every row. Restoring is done by importing the file into MySQL (phpMyAdmin → Import, or the `mysql` command), which replaces all current data with the data in the backup. Password hashes are included, so restored accounts keep their passwords; plain passwords are never in it, and neither is the

pepper, because the pepper is not stored in the database. Only administrators can download a backup, and every download is logged.

Restore points. A restore point is different from a backup: it saves only the *configuration*, that is, each account's role and whether it is deleted. Applying a restore point puts that configuration back and changes nothing else: names, emails and password hashes stay as they are, and accounts created after the restore point are left alone. This lets an administrator undo, for example, a wrong role change or a mistaken deletion without touching any data. To avoid locking themselves out, the server refuses to apply a restore point that would remove the administrator's own admin access. Sessions of accounts that the restore point marks as deleted are ended.

12. Reports and the password demonstration

Event log and reports. The server records security events in the `auth_events` table: registrations, correct passwords, failed logins (an unknown email is stored masked, for example `n***@example.com`), wrong codes, successful logins, logouts and every administrator action (delete, restore, backup, restore points). Passwords and codes are never written to the log. The Reports page shows the number of accounts by role and status, active sessions, the number of restore points and the time of the last backup; successful logins per day for the last seven days; the count of each event type; and the 25 most recent events with time, account, detail and IP address. In the browser the page can be printed or saved as PDF.

Password demonstration. To show salt and pepper working, the Password demo page (administrators only) takes any sample password and shows each step: the password combined with the pepper using HMAC-SHA256 (only a shortened result is shown and the pepper itself never leaves the server); the random salt; the final Argon2id hash; a second hash of the same password with a different salt and therefore a different result; and three checks with `password_verify()`: the correct password is accepted, a wrong password is rejected, and the correct password *without the pepper* is rejected. Nothing typed on this page is saved or logged.

13. Implementation details

Area	Where
Entry point and routing, CORS, CSRF, role guard	`backend/src/Http/Kernel.php`, `backend/public/index.php`
Registration, login, OTP, sessions	`backend/src/Services/AuthService.php`

Area	Where
Deleting accounts, backup, restore points, reports, password demo	`backend/src/Services/AdminService.php`
Event log	`backend/src/Services/AuditLog.php`
Salt and pepper	`backend/src/Security/Passwords.php`
Secure random values	`backend/src/Security/Tokens.php`
Rate limiting	`backend/src/Security/RateLimiter.php`
Email (SMTP) and development delivery	`backend/src/Mail/`
Validation	`backend/src/Validation/Validator.php`, `mobile/src/lib/validation.ts`
Configuration safety checks	`backend/src/Config.php`
Database schema and upgrade	`backend/database/schema.sql`, `backend/database/migrations/`
Command-line tools	`backend/bin/setup-db.php`, `set-role.php`, `generate-secret.php`
App screens	`mobile/app/`
API client, session state, secure storage	`mobile/src/lib/api.ts`, `auth.tsx`, `storage.ts`

The interface is deliberately simple: a header bar with the application name and (when logged in) the menu links Dashboard, Profile and Logout, plus Users, Backup, Reports and Password demo for administrators, with the page content below it. Forms are a single column with a maximum width, so the same layout works on a phone and in a desktop browser. The admin user list is a plain table that scrolls sideways on narrow screens. Forms show a loading state on the button, field-level errors and readable messages.

14. Testing

14.1 Automated backend tests (PHP)

`backend/tests/run.php` contains 74 tests that call the API kernel directly against a separate test database (`login_auth_test`). They cover every item required by the brief:

Requirement	Test file
Registration succeeds; duplicates (including different letter case) are rejected; validation; SQL-injection input	`registration.php`
Passwords are stored as salted, peppered hashes (plaintext absent, wrong pepper fails, same password gives different hashes)	`registration.php`

Requirement	Test file
Correct and incorrect password; identical generic errors for unknown email	`login_and_otp.php`
OTP success, wrong code, attempt limit, expiry (exactly 300 s), reuse, replacement, resend, delivery failure; hashes only in the database	`login_and_otp.php`
Session creation, invalid and expired sessions, logout and revocation, web cookie flow and CSRF checks	`login_and_otp.php`
Admin can list users; normal user gets 403; anonymous gets 401; role comes from the database; no hashes in output	`authorization.php`
CORS (allowed origin, rejected origin, never `*`), 404/405/400/413/415, security headers, rate limits (account, IP, registration, OTP guessing)	`http_and_abuse.php`
Configuration errors handled safely (missing/short/placeholder pepper, dev OTP in production, wildcard CORS, insecure cookies, bad SMTP, database down) and no dev OTP in production responses	`configuration.php`
Every admin endpoint refuses students (403) and visitors (401); deleting marks the account, ends its sessions and blocks login; restoring; no self-delete; CSRF headers on cookie actions; restore points change only roles and deleted status; full backup contents; reports counts; password demo (different salts, pepper never shown, nothing saved)	`admin_tools.php`

How the tests were run. On the development computer, Windows Smart App Control blocked `php.exe`, and no MySQL server could be started. The suite was therefore run with `tools/wasm-runner` (`npm test`): PHP 8.3 compiled to WebAssembly, running inside Node.js, with an in-memory **SQLite** database instead of MySQL. A small translation layer (`tools/wasm-runner/compat.php`) rewrites the few MySQL-only statements (`ON DUPLICATE KEY UPDATE`, `TRUNCATE`, `SET FOREIGN_KEY_CHECKS`, one `DATE_ADD` in a test). The application code itself is not changed. Because this PHP build has no Argon2 support, the bcrypt fallback was the algorithm exercised.

Result:

```
74 passed, 0 failed (1066 assertions)
```

To check that the tests really detect faults, three bugs were deliberately introduced in a copy of the code, one at a time. Each was caught: removing the admin role check failed 2 authorization tests; making codes valid for 99,999 seconds failed the five-minute expiry test; and no longer marking codes as used failed 4 OTP tests (including "a code can be used only once"). The same was done for the new features: letting deleted accounts keep their sessions, letting a restore point change names, removing the CSRF check on admin actions and showing the full peppered value in the demo were each caught by the new tests. (The first of these was at first *not* caught, because deleting an account also ends its sessions; a

test that marks an account as deleted directly in the database was added, and it now fails as it should.)

Not yet verified against MySQL. The MySQL schema (`database/schema.sql`), the MySQL form of the rate-limit query and the duplicate-key error code (1062) were not exercised by this run. The schema and the rate-limit query were later used on the online MySQL database (section 14.4), but the test suite itself has not run against MySQL. On a computer where XAMPP or Laragon works, run:

```
cd backend
php tests/run.php
```

and add the output here:

```
(paste the output of php tests/run.php here, including the final "N passed,
M failed" line)
```

14.2 Client checks that were actually run

Check	Command	Result
TypeScript type check (strict mode)	`npm run typecheck` in `mobile/`	Completed with no errors
Client validation unit tests	`npm test` in `mobile/`	6 tests, 6 passed, 0 failed
Production web build	`npx expo export --platform web`	Succeeded; output in `mobile/dist` (one JavaScript bundle, about 1.1 MB, and `index.html`)
Secrets in build output	search of the built JavaScript for `DB_PASS`, `APP_PEPPER`, `SMTP_PASS`	None found

14.3 End-to-end tests over HTTP and in a browser

The unit tests above call the API classes directly, so they skip the code that reads real HTTP requests and writes real responses (headers, cookies, `public/index.php`). Two further checks covered that path, using the real PHP API served by `tools/wasm-runner` (`npm run api`), again with SQLite instead of MySQL:

- **HTTP checks (26 passed, 0 failed):** register, duplicate and invalid registration, wrong password, login with code, wrong code with attempts remaining, correct code, code reuse refused, bearer-token session, normal user refused on the admin route (403), anonymous refused (401), logout and revocation, CORS for allowed and unknown origins, a browser sign-in from a disallowed origin refused, the `HttpOnly; SameSite=Lax` session cookie, promotion with the real `bin/set-role.php`, the

admin user list without password hashes, and cookie logout refused without the CSRF headers.

- **Browser test (72 passed, 0 failed):** the web build was opened in a headless Edge browser and driven through a script in three sessions (desktop student, phone-sized administrator, desktop administrator) against the real PHP API. Besides the login checks below, it deletes and restores an account on the Users page, downloads a full backup, saves and applies a restore point, opens the Reports page, runs the Password demo, and confirms that a student is refused (403) on the Users, Backup, Reports and Password demo pages.

While these were being set up, a database-locking error appeared. It was caused by the test environment (SQLite file locks on a mounted Windows folder), not by the application, and was fixed in the runner. Some early runs also failed one step each because the script checked the page before the server had finished hashing the password; the script was changed to wait for each result. No bugs were found in the PHP application code.

Earlier, before the backend could be run, the same browser script was used against a small mock API to develop the interface. The browser checks were: redirect to sign-in when signed out; empty and weak inputs rejected on the client; registration; generic error for a wrong password; moving to the code screen; wrong then correct code; dashboard and role display; session survives a reload through the cookie; no token in `localStorage` or `sessionStorage`; the session cookie is not readable from JavaScript; profile page; the administration page listing users for an administrator and showing "Access denied (403)" for a normal user; sign-out and redirect. Two bugs found by this process (a confusing error when the API address was wrong, and a spurious "session ended" message at start-up) were fixed.

14.4 Tests of the online version (real MySQL and real email)

The deployed site at https://puit24210065.vercel.app was tested on 11 October 2026.

- **Database setup:** `backend/database/schema.sql` was run unchanged on Aiven MySQL 8.4.11 over a verified TLS connection and created the four tables. This is the first run of the project against real MySQL.
- **Automated live checks (17 passed, 0 failed):** health check reaching the database; `Strict-Transport-Security` and `no-store` headers; registration (a new account gets the lowest role); duplicate email in different letter case refused (409); invalid input refused (422); wrong password refused with the generic message (this also exercises the MySQL form of the rate-limit query); correct password sends the code **by email**, and the response does not contain the code; masked email hint; a wrong code is refused with 4 attempts left; no session without the code (401); the admin route refused without

a session (401); CORS allows only the site's own origin; finishing a sign-in from another origin is refused (403).

- **Manual check:** an account was registered on the public site with a personal Gmail address, the emailed code arrived and was entered, and the account was signed in. That account was then made an administrator directly in the database (the online equivalent of `bin/set-role.php`).
- **Second round (18 October 2026), after adding the administrator features:** before upgrading, a copy of all live data was saved; then the upgrade script `002_student_role_backup_reports.sql` was applied to the live database and the new version was deployed. The 17 live checks passed again. A temporary administrator account was then used for 12 more live checks, all passed: the emailed code was accepted; the user list showed the Argon2id preview; a restore point was saved; a test account was deleted and could no longer log in; applying the restore point brought it back (configuration only); a full backup was downloaded; the reports counted these actions; the password demo used Argon2id, gave two different salts and refused the password without the pepper; and the restored account could log in again. Finally the backup file was **imported into a separate, empty MySQL database**, and every table had exactly the same number of rows as the original, which shows the backup can really be restored. The temporary accounts were removed afterwards.
- **Problem found and fixed:** the first live run failed to send email (SMTP error 535, "username and password not accepted") because the App Password had been created in a different Google account from the sender address. The application handled it safely: it answered 503 and kept no code. Using the matching account fixed it.

14.5 Screenshots

Figures 1–10 and 13–16 were captured from the web version of the app running against the real PHP API on the local computer (`tools/wasm-runner`, development mode, so the code is shown on screen instead of being emailed; that PHP build has no Argon2, so Figure 16 shows bcrypt). Figures 11 and 12 are from the online MySQL database.

Figure 1. Login screen.

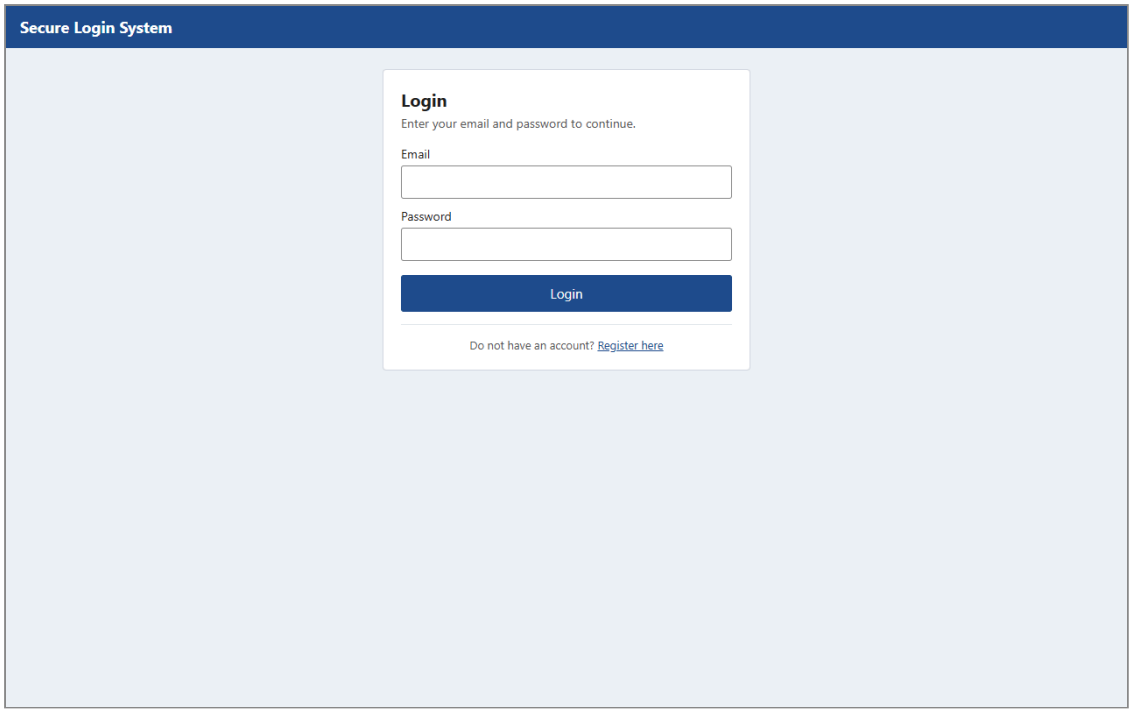


Figure 2. Registration with client-side validation errors.

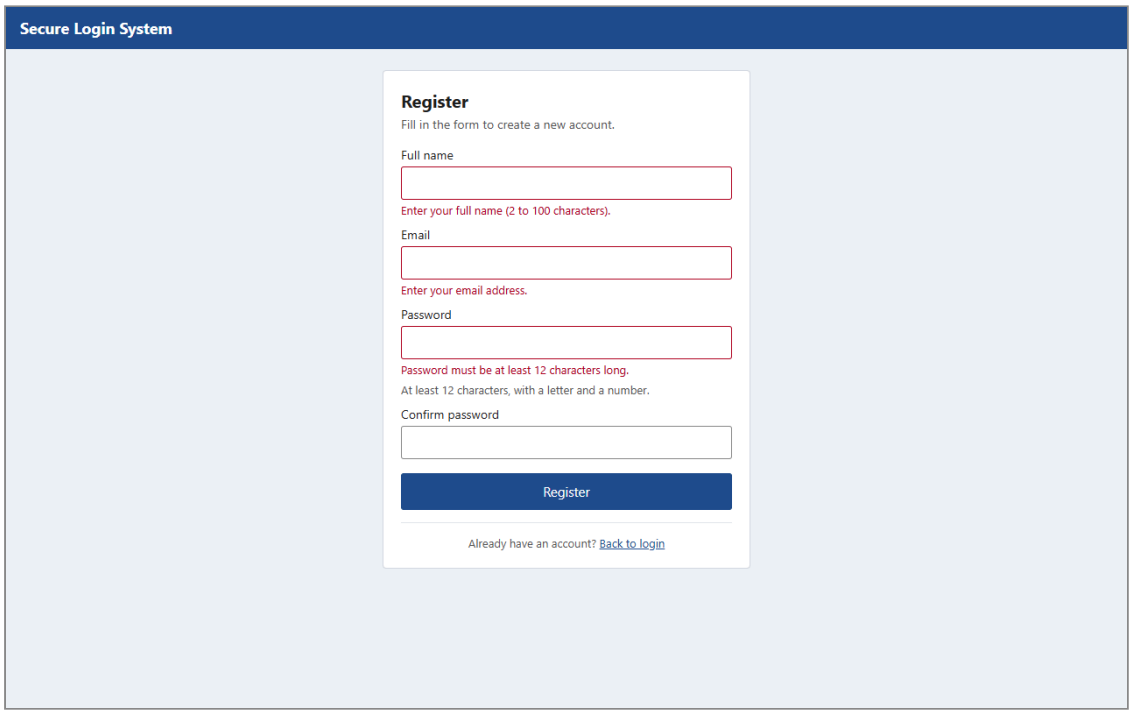


Figure 3. After a successful registration the user is returned to the login screen with a message.

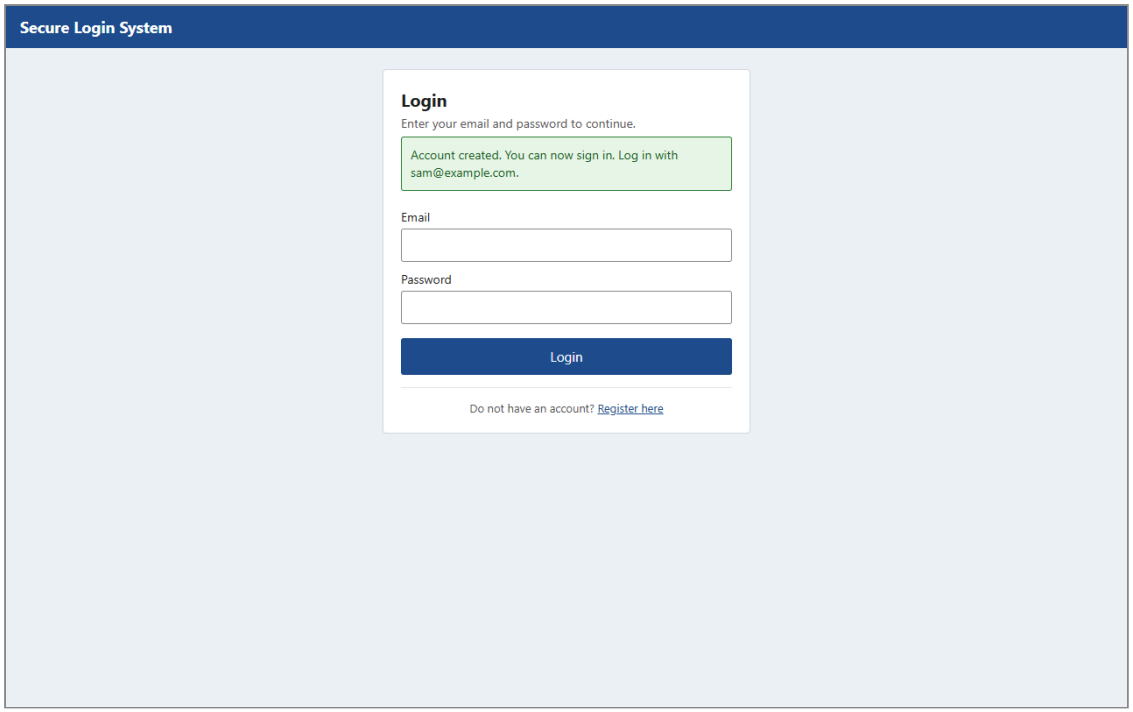


Figure 4. Two-factor step after the correct password. In development mode the code is shown in the yellow box; online it is emailed.

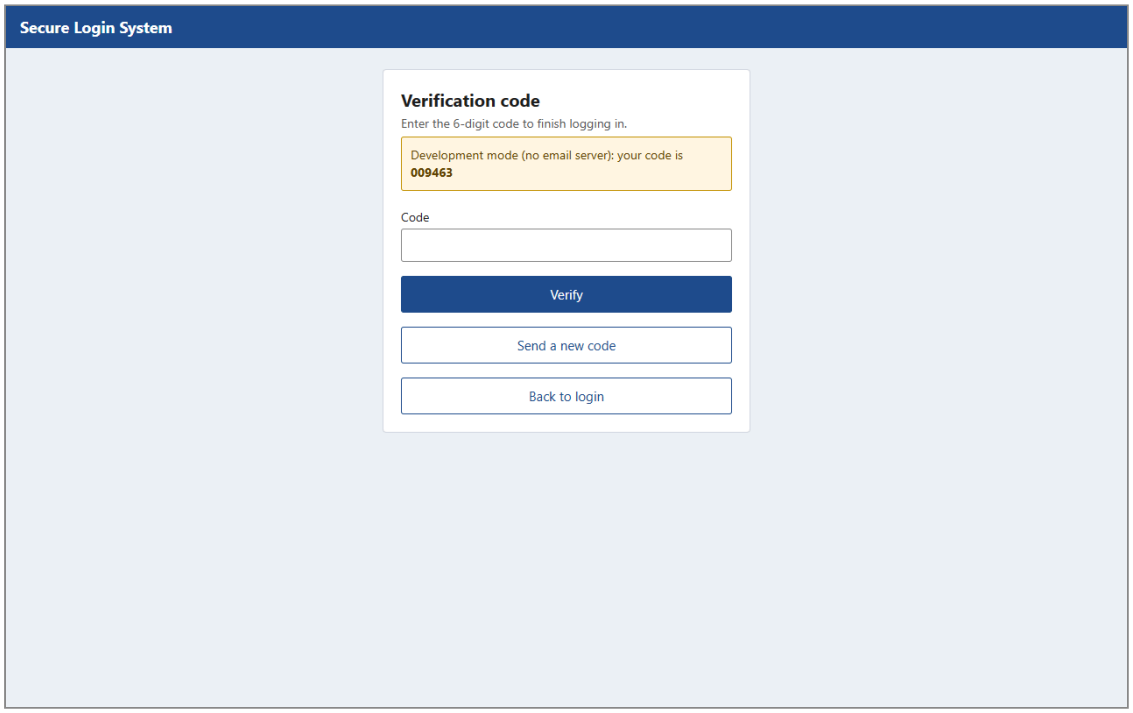


Figure 5. A wrong code is refused (the server also counts the attempt).

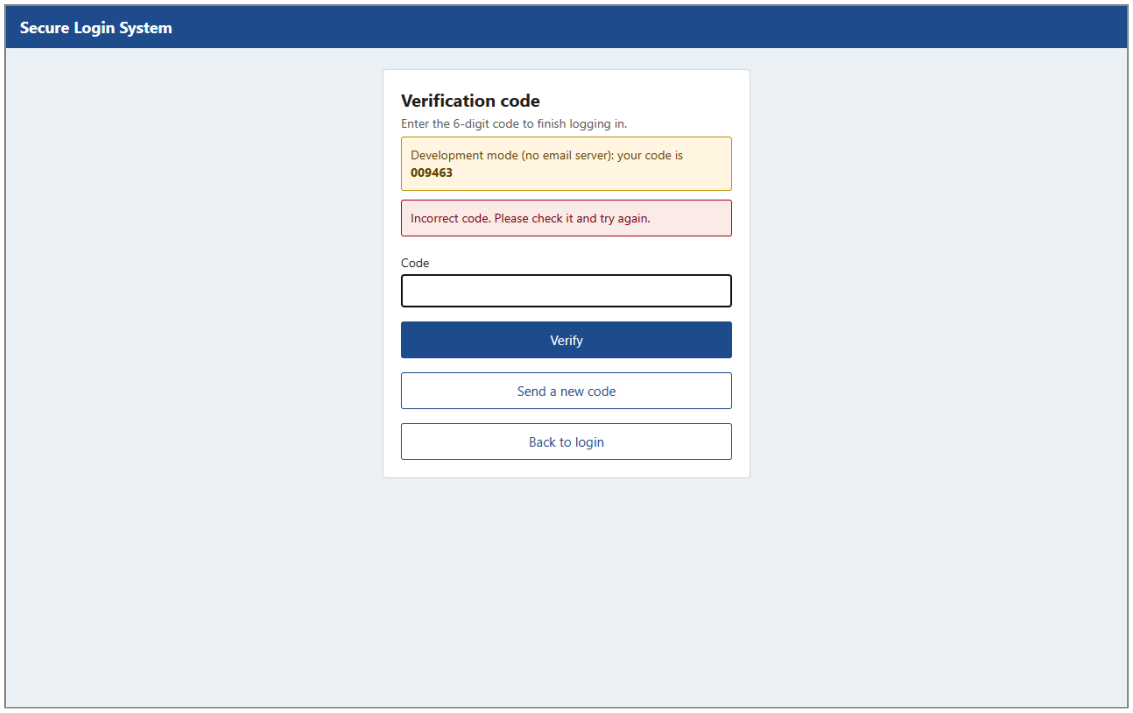


Figure 6. Dashboard of a standard user after password + code.

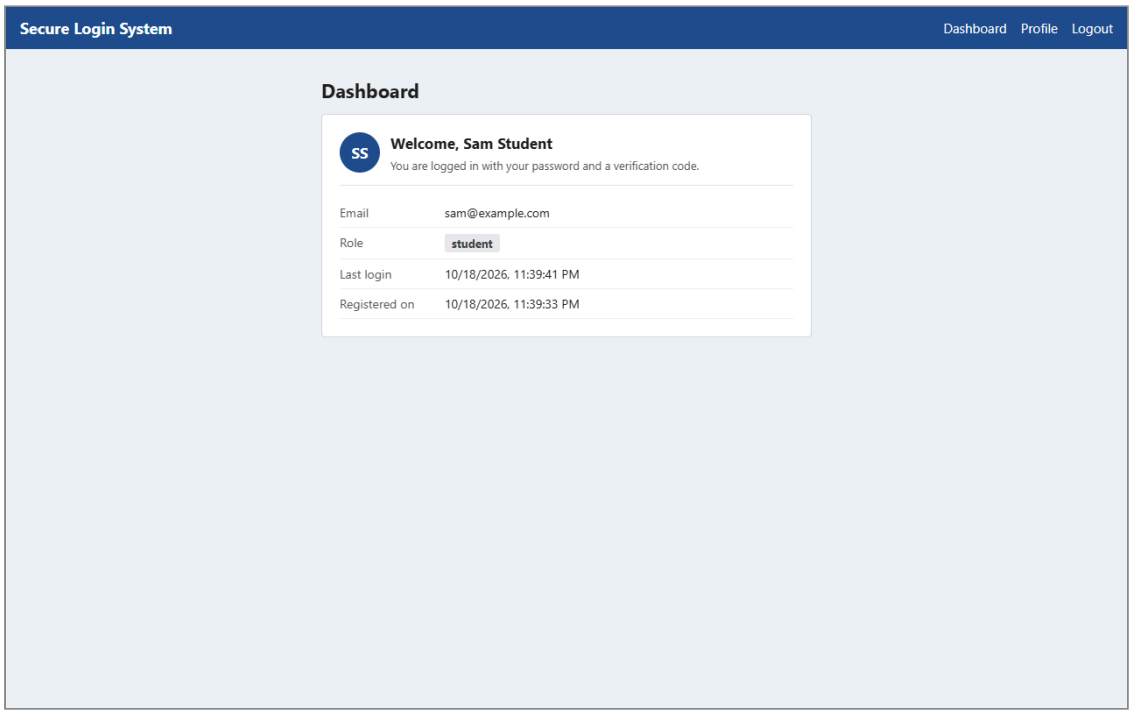


Figure 7. Profile page showing the role.

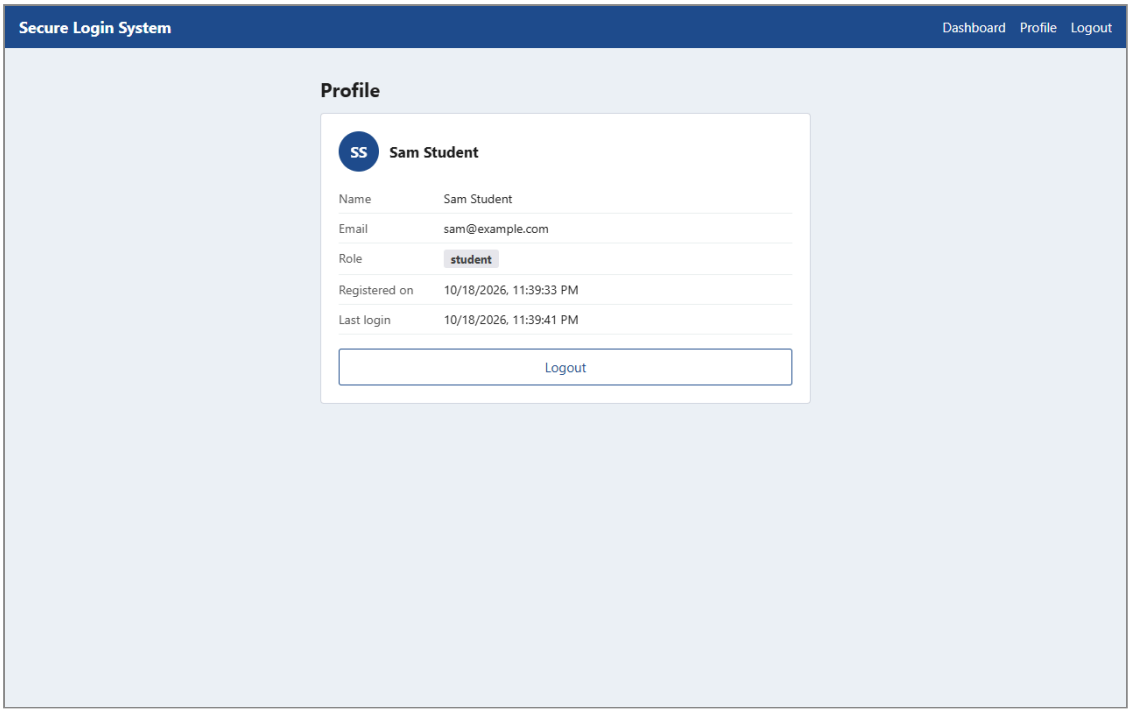


Figure 8. A standard user who opens the admin page by typing its address is refused by the server (403).

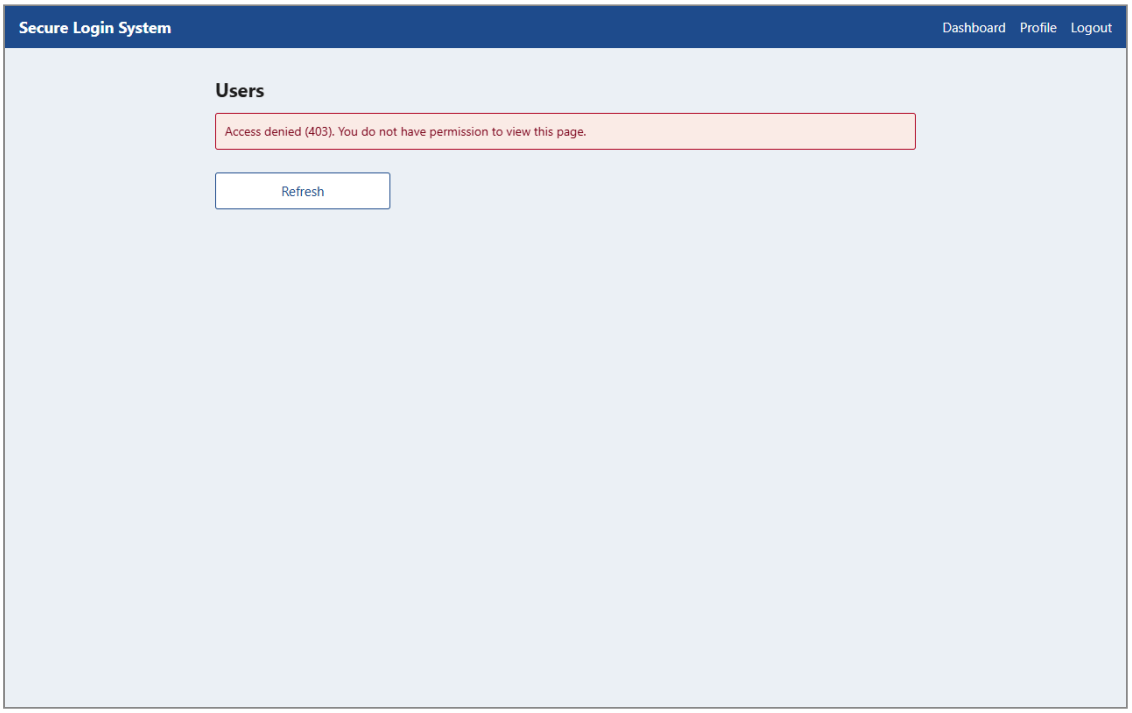


Figure 9. The Users page for an administrator. Only a shortened preview of each stored hash is sent.

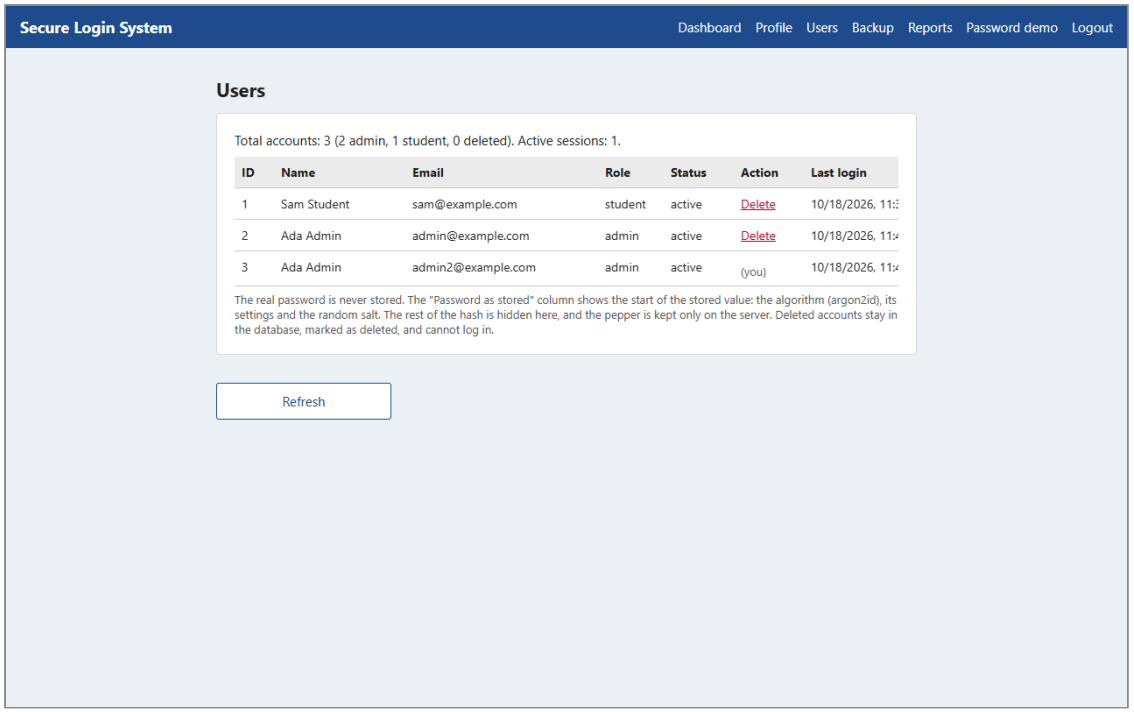


Figure 10. The dashboard at phone width.

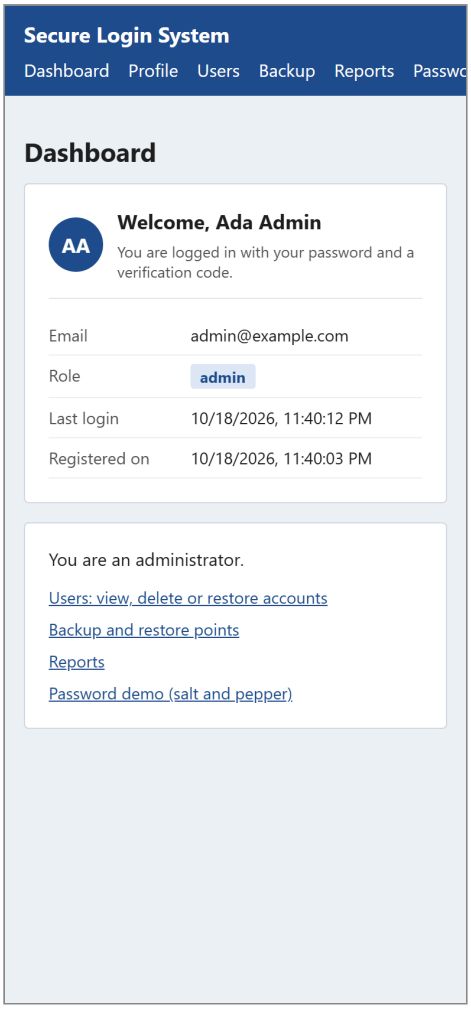


Figure 11. The `users` table in the online MySQL database. The password column contains only the Argon2id hash produced by `password_hash()`; the salt is inside the hash and the pepper is not stored.

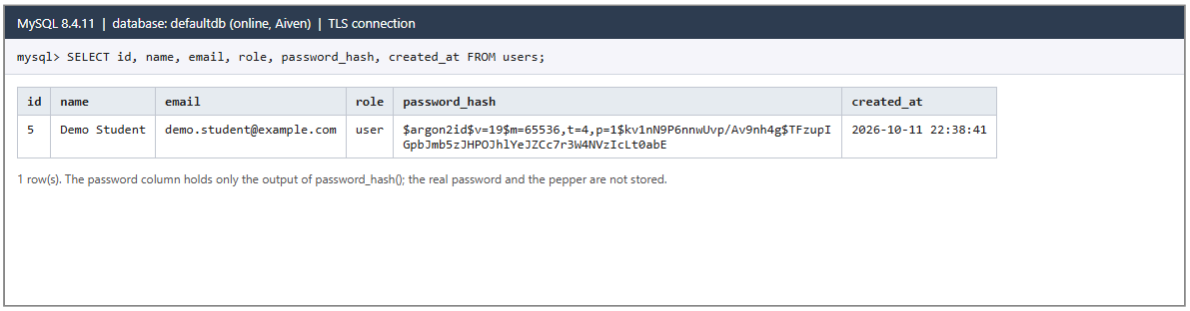
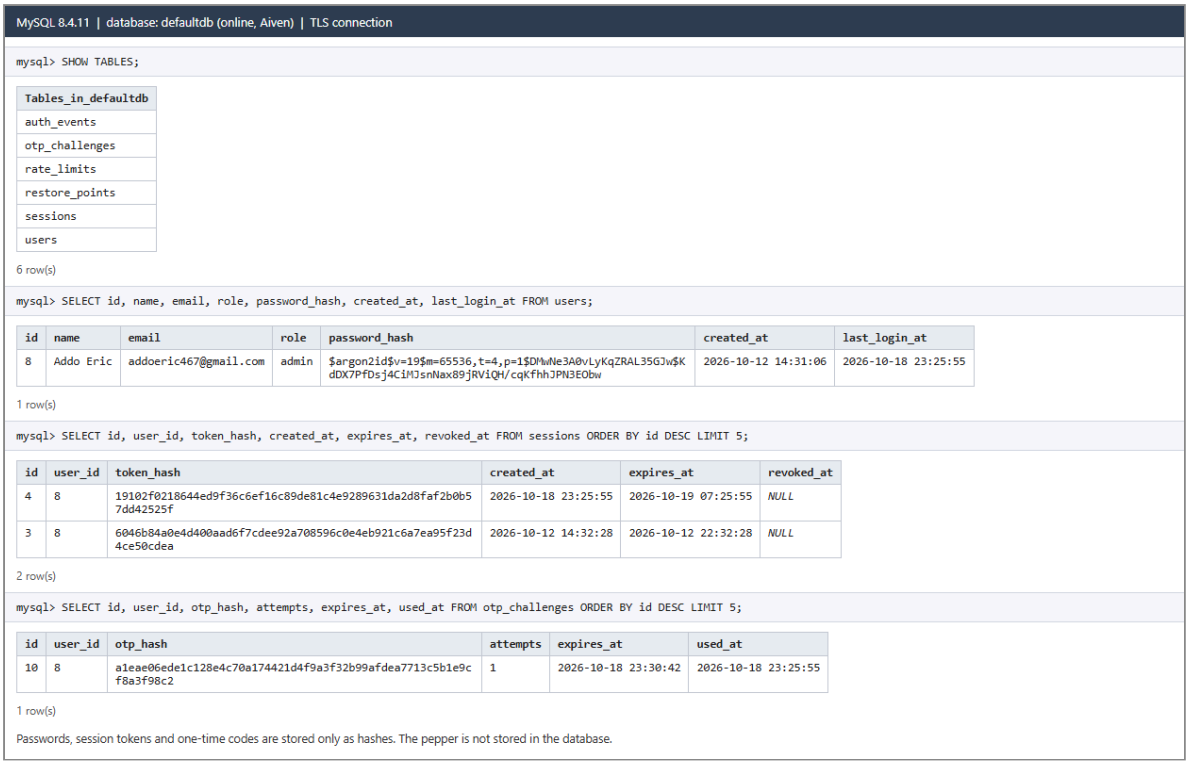


Figure 12. The live database: all six tables, with passwords, session tokens and codes stored only as hashes.



MySQL 8.4.11 | database: defaultdb (online, Aiven) | TLS connection

```
mysql> SHOW TABLES;
```

Tables_in_defaultdb
auth_events
otp_challenges
rate_limits
restore_points
sessions
users

6 row(s)

```
mysql> SELECT id, name, email, role, password_hash, created_at, last_login_at FROM users;
```

id	name	email	role	password_hash	created_at	last_login_at
8	Addo Eric	addoeric467@gmail.com	admin	$argon2id$v=19$m=65536,t=4,p=1$DMwNe3A0vLyKqZRAL35GJw$KdDX7PfDsj4CiMJsnNax89jRViQH/cqKfhhJPN3EObw	2026-10-12 14:31:06	2026-10-18 23:25:55

1 row(s)

```
mysql> SELECT id, user_id, token_hash, created_at, expires_at, revoked_at FROM sessions ORDER BY id DESC LIMIT 5;
```

id	user_id	token_hash	created_at	expires_at	revoked_at
4	8	19102f0218644ed9f36c6ef16c89de81c4e9289631da2d8faf2b0b57dd42525f	2026-10-18 23:25:55	2026-10-19 07:25:55	*NULL*
3	8	6046b84a0e4d400aad6f7cdee92a708596c0e4eb921c6a7ea95f23d4ce50cdea	2026-10-12 14:32:28	2026-10-12 22:32:28	*NULL*

2 row(s)

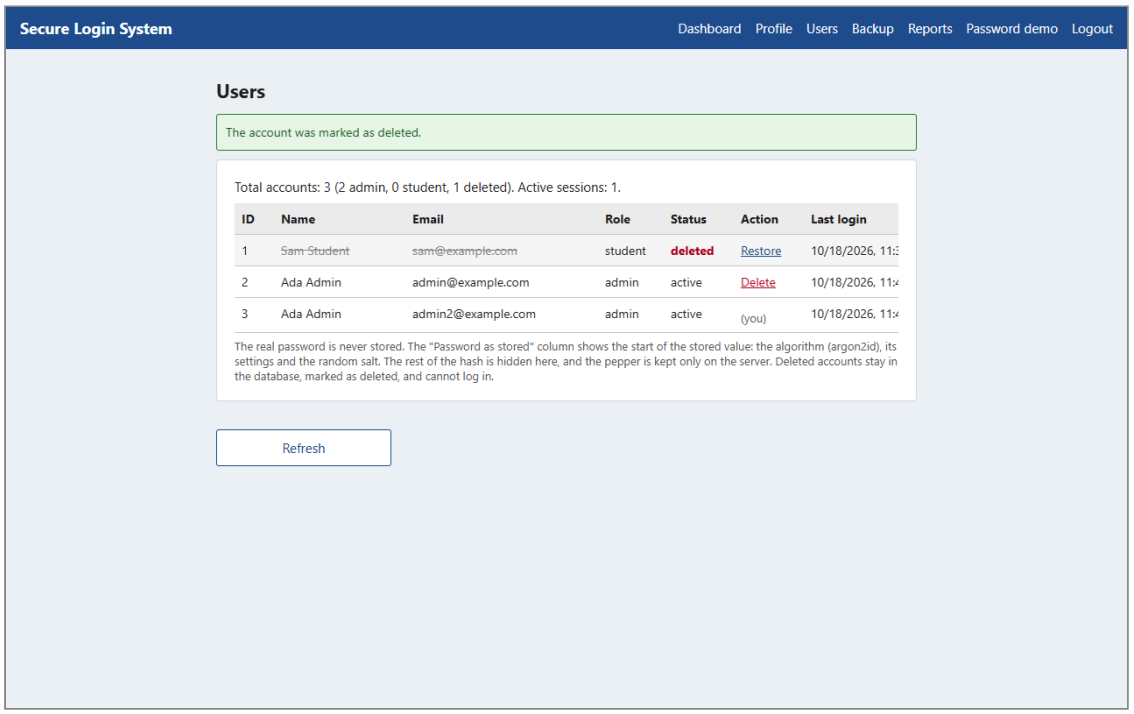
```
mysql> SELECT id, user_id, otp_hash, attempts, expires_at, used_at FROM otp_challenges ORDER BY id DESC LIMIT 5;
```

id	user_id	otp_hash	attempts	expires_at	used_at
10	8	a1eae06ede1c128e4c70a174421d4f9a3f32b99afdea7713c5b1e9cf8a3f98c2	1	2026-10-18 23:30:42	2026-10-18 23:25:55

1 row(s)

Passwords, session tokens and one-time codes are stored only as hashes. The pepper is not stored in the database.

Figure 13. An account marked as deleted: it stays in the list (struck through) and can be restored.



Secure Login System — Dashboard | Profile | Users | Backup | Reports | Password demo | Logout

Users

The account was marked as deleted.

Total accounts: 3 (2 admin, 0 student, 1 deleted). Active sessions: 1.

ID	Name	Email	Role	Status	Action	Last login
1	~~Sam Student~~	~~sam@example.com~~	student	deleted	Restore	10/18/2026, 11:
2	Ada Admin	admin@example.com	admin	active	Delete	10/18/2026, 11:
3	Ada Admin	admin2@example.com	admin	active	(you)	10/18/2026, 11:

The real password is never stored. The "Password as stored" column shows the start of the stored value: the algorithm (argon2id), its settings and the random salt. The rest of the hash is hidden here, and the pepper is kept only on the server. Deleted accounts stay in the database, marked as deleted, and cannot log in.

Refresh

Figure 14. Backup page: full database backup (download) and restore points (configuration only).

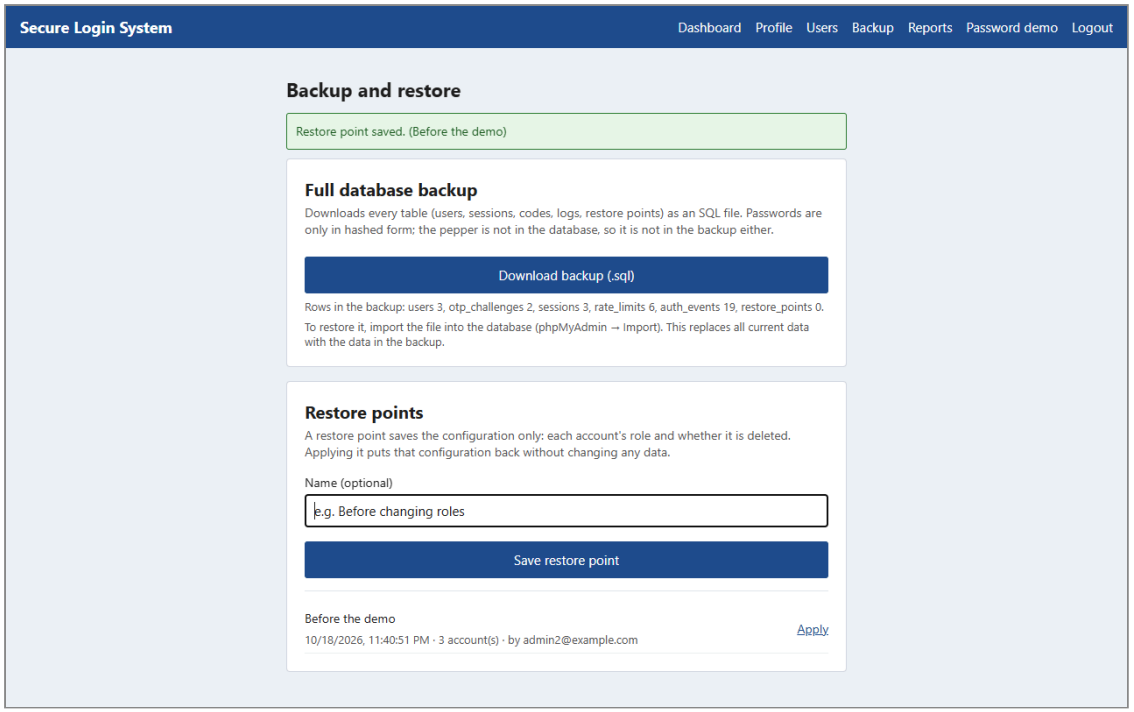


Figure 15. Reports page: accounts and successful logins per day.

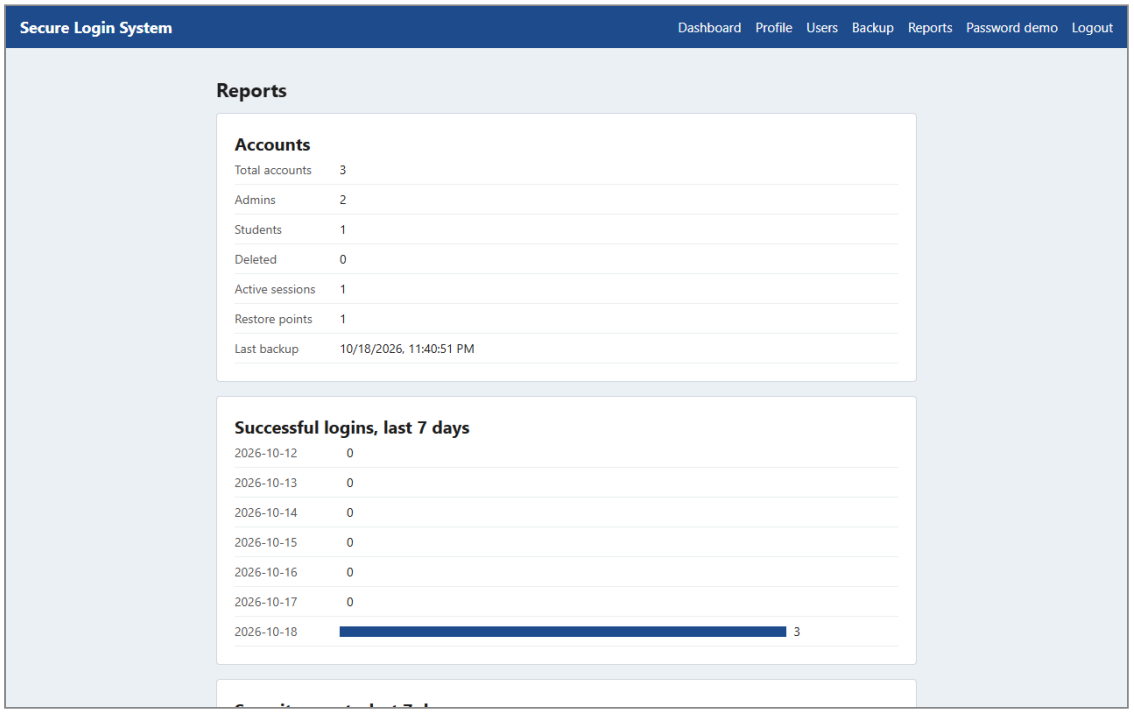
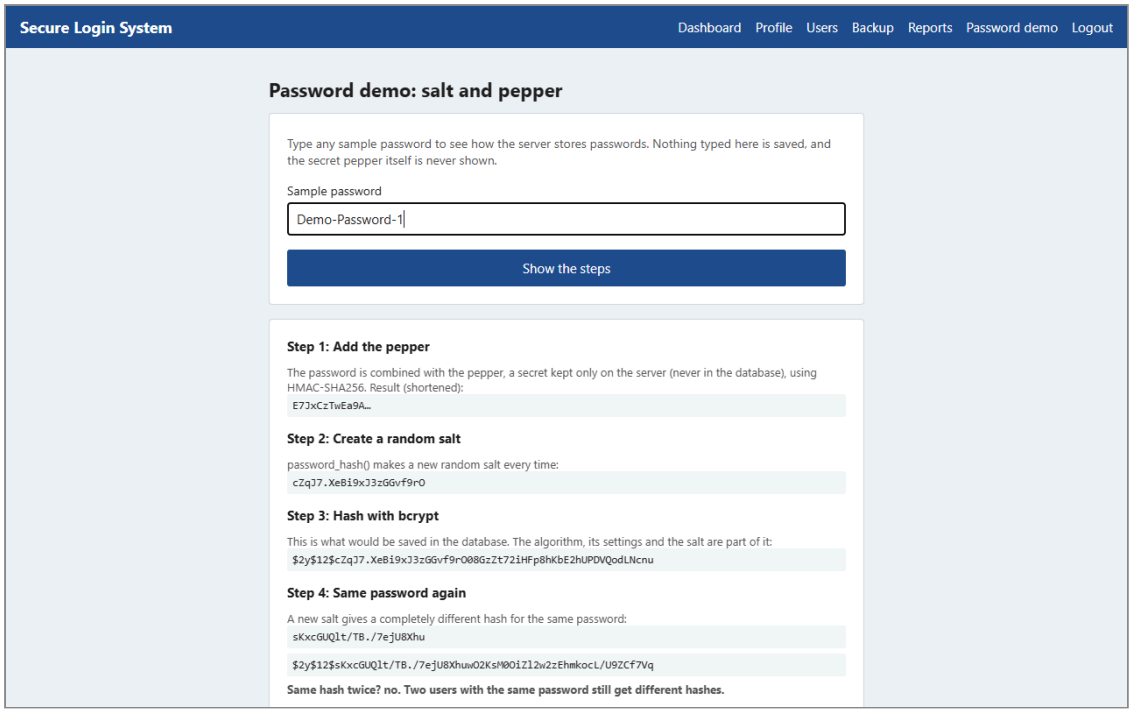


Figure 16. Password demo: the pepper step, two different salts and hashes for the same password.



Still to add from a local XAMPP or Laragon run: the terminal output of `php tests/run.php`, and the app running in Expo Go on a phone.

15. Limitations and future improvements

- The backend tests were run with PHP in WebAssembly against SQLite, not yet against MySQL (section 14.1).
- The pepper is a single secret; rotating it would require a re-hash migration (for example storing a pepper version with each hash) or forced password resets.
- Account lockout is per email, so an attacker can temporarily lock a victim's login by sending wrong passwords. A CAPTCHA or progressive delays would reduce this.
- Registration reveals whether an email is already registered (needed for a clear error message). Email verification would avoid this.
- No password reset, email verification or "remember this device" feature; sessions are fixed-length.
- Roles are changed only from the command line or the `ADMIN_EMAILS` setting; there is no page for changing roles.
- Restoring a full backup is done in phpMyAdmin or with the `mysql` command, not from the app, and it replaces all data. Restore points cover only roles and deleted status.
- The event log keeps growing; old entries are not yet archived or deleted automatically.
- The second factor is an emailed code, which is only as secure as the mailbox. Authenticator apps (TOTP) or WebAuthn passkeys are stronger.

- Rate limits are stored in MySQL and keyed by IP address, which can be inaccurate behind shared networks or proxies.
- The online version depends on three free or trial services (Vercel, Aiven, Gmail). Gmail limits sending to about 500 emails a day, codes can land in spam folders, and the first request after a quiet period can be slow while Vercel starts the PHP function.

16. Conclusion

The project implements the four required elements: users are identified through validated registration and a password check; a second factor (a single-use, expiring, attempt-limited one-time code) is required before a session is created; authorization depends on a role that the server reads from the database on every protected request; and passwords are protected with a per-password salt plus a server-side pepper. Sessions are random, stored only as hashes, expire and can be revoked, and the web version avoids storing tokens where scripts can read them. Administrators can also mark accounts as deleted without losing their data, download a full database backup, save and apply restore points that change only the configuration, view reports built from a security event log, and demonstrate salting and peppering step by step.

The interface was built, type-checked, unit-tested and compiled for the web. The backend's 74 tests pass, and the whole system was tested end to end over HTTP and in a browser with the real PHP code, using SQLite in place of MySQL. The online version was then tested against real MySQL with real emailed codes, and sign-in works there. The full test suite has still to be run against a local XAMPP or Laragon MySQL.

17. Instructions for running the application

See `README.md` for the full step-by-step guide. In short, on Windows with XAMPP and Node.js installed:

```
# 1. start MySQL in the XAMPP Control Panel
cd backend
copy .env.example .env          # then put the output of  php bin/generate-
secret.php  after APP_PEPPER=
php bin/setup-db.php            # new database; an older one is upgraded
with database/migrations/002_*.sql
php tests/run.php               # backend tests
php -S localhost:8000 -t public public/index.php

# in a second terminal
cd mobile
npm install
npm test
npx expo start                  # press w for the browser
```

To make an administrator: register in the app, then `php bin/set-role.php you@example.com admin`. The online version is at https://puit24210065.vercel.app. How it is set up (Vercel, Aiven MySQL, Gmail) is described in `docs/DEPLOYMENT.md`, section 0.